MATH36031 Project 1

Audrey Ward

October 2026

1 Introduction

Decimal notation has a long history, with mathematicians gradually developing ways to represent fractions more conveniently. In 1585, the Flemish mathematician Simon Stevin published De Thiende [1], a work that helped establish the systematic use of decimal fractions in Europe. Although decimal fractions had been used earlier in other mathematical traditions, Stevin's work contributed to their wider adoption.

Despite the simplicity of decimal notation, fractions do not always produce terminating decimal expansions. Some fractions instead have recurring decimal digits, forming repeating patterns whose lengths depend on the properties of the numerator and denominator. Identifying these patterns provides a connection between decimal representation and number theory. This project investigates this problem computationally by implementing two algorithms in MATLAB to calculate recurring digits and determine their periods, before searching for fractions with the longest recurring period under the given constraints.

2 Algorithm 1

2.1 Mathematical Method

Consider a fraction p/q, where p and q are positive integers satisfying $p < q$, and the last digit of q is 9. The decimal expansion can be generated iteratively using integer arithmetic. Let p_n denote the remainder at iteration n, with initial remainder $p_0 = p$. The denominator q remains fixed throughout the algorithm.

At each iteration, the next decimal digit is calculated as

$$a_{n+1} = \left\lfloor \frac{10p_n}{q} \right\rfloor, \tag{1}$$

where $\lfloor x \rfloor$ denotes the floor function, which returns the greatest integer less than or equal to x. Multiplying the current remainder by 10 advances the long-division process to the next decimal place, while taking the floor extracts the next digit.

The remainder is then updated according to

$$p_{n+1} = 10p_n - a_{n+1}q. \tag{2}$$

This update subtracts the contribution of the digit just obtained, leaving the remainder required for the next iteration. Repeating these two steps generates the decimal digits sequentially. The period length k is determined by counting the digits generated before the stopping condition is met.

2.2 Worked Example

Consider the fraction 18/39. Initially, $p_0 = 18$ and $q = 39$. The first digit is

$$a_1 = \left\lfloor \frac{10(18)}{39} \right\rfloor = 4, \tag{3}$$

giving the new remainder $p_1 = 10(18) - 4(39) = 24$.

Using this remainder, the second digit is

$$a_2 = \left\lfloor \frac{10(24)}{39} \right\rfloor = 6, \tag{4}$$

and the remainder becomes $p_2 = 10(24) - 6(39) = 6$.

Applying the same procedure repeatedly generates the digits $4, 6, 1, 5, 3, 8$. Therefore,

$$\frac{18}{39} = 0.\overline{461538}, \tag{5}$$

and the period length is $k = 6$.

2.3 Code Implementation and Validation

The mathematical method described above was implemented in MATLAB using the function `RecFrac1` as shown in A.1, which takes p and q as inputs and returns the period length k and the vector of recurring digits a.

Before performing the calculations, the function validates the inputs. Both inputs must be positive, finite, real integers represented by scalar numeric values. The function also checks that $p < q$ and that the final digit of q is 9. If any of these conditions is violated, an error is produced.

The initial value of the numerator is stored in `p0`, which is used to identify when the remainder returns to its starting value. An initially empty vector `a` is used to store the decimal digits generated by the algorithm. A `while` loop then repeatedly calculates the next digit using `floor(10*p/q)`, appends this digit to the vector, and updates the remainder using `p = 10*p - digit*q`.

The loop terminates when the updated remainder equals `p0`. Finally, the period length is calculated using `k = length(a)`, which counts the number of digits stored in the vector. The function therefore returns both the recurring digits and their period length.

The function was also tested using valid inputs with known recurring decimal expansions to verify that both the digit vector and period length were returned correctly. Additional tests were performed using invalid inputs, including non-positive values, non-integer values, cases where $p \geq q$, and denominators that do not end in 9. These tests checked that the function rejected inputs that violated the specified conditions.

3 Algorithm 2

3.1 Mathematical Method

Algorithm 2 generates the recurring decimal digits using repeated integer division. First, a divisor d is obtained from the denominator q by removing its final digit, which is 9, and adding 1. Hence,

$$d = \frac{q - 9}{10} + 1.$$

For example, when $q = 39$, the divisor is $d = 4$.

Starting with $n = p$, each iteration calculates the next digit by taking the integer quotient when n is divided by d. The remainder is then used to construct the next number for the following iteration. If the remainder is zero, the next number is the quotient digit itself; otherwise, the remainder is placed before the quotient digit. This process generates the digits sequentially.

When a number is encountered for a second time, the cycle has returned to a previous state, so we know the digits would start repeating, hence the algorithm terminates.

3.2 Code Implementation and Validation

The algorithm was implemented in MATLAB as the function `RecFrac2`, as seen in A.2, which takes p and q as inputs and returns the period length k and the vector of recurring digits a. Similarly to algorithm 1 2, input validation is performed to ensure that both inputs are positive integers, that $p < q$, and that q ends in 9. If any of these conditions is violated, an error is produced.

The divisor is calculated using `d = (q - 9)/10 + 1`. The function initialises `n` to p and creates two empty vectors: `a`, to store the digits generated, and `seen`, to record the numbers encountered during the algorithm. A `while` loop uses `ismember` to check whether the current number has already appeared in `seen`. If it has not, the number is recorded and the next digit and remainder are calculated using `floor` and `mod`, respectively.

The digit is appended to `a`, and the remainder and digit are used to calculate the next value of `n`. The loop terminates when a previously encountered number is reached. Finally, `k = length(a)` calculates the number of digits generated, giving the period length returned by the function.

The code was also tested and validated in the same way as algorithm 1.

4 Comparison of the Two Algorithms

The two algorithms produce the same recurring digits and period lengths, but differ in how they detect the end of the repeating cycle. Algorithm 1 terminates when the remainder returns to its initial value, whereas Algorithm 2 stores previously encountered states and uses `ismember` to detect repetition. This difference affects their computational efficiency.

The execution times were compared using MATLAB's `timeit` function. For the fraction $\frac{18}{39}$, which has a period of 6 digits, Algorithm 1 took 3.814×10^{-5} seconds, while Algorithm 2 took 2.314×10^{-5} seconds. Thus, Algorithm 2 was approximately 1.65 times faster for this example. However, for $\frac{708}{709}$, which has a period of 708 digits, Algorithm 1 took 1.128×10^{-4} seconds, compared with 7.749×10^{-4} seconds for Algorithm 2. Algorithm 1 was therefore approximately 6.87 times faster for the longer period.

These results suggest that Algorithm 2 can perform well for short periods, but its repeated searches through the `seen` vector become more costly as the number of iterations increases. Algorithm 1 uses a simpler stopping condition and does not need to search through a growing vector of previous states. Nevertheless, these measurements concern only two test cases, so further experiments would be needed to establish a general performance trend.

Overall, neither algorithm is universally faster in every situation. Algorithm 1 was more efficient for the longer test case, while Algorithm 2 was faster for the shorter one. The results illustrate how the choice of algorithm and its implementation can affect computational performance, even when both methods produce the same mathematical result.

5 Search for Maximum Period

We now aim to search for the maximum period length among fractions $\frac{r}{s}$ satisfying the given constraints, where $r < s \leq 749$ and s ends in 9. For each eligible denominator s, the possible numerators r are considered, and the period length is calculated using the algorithms previously developed in sections 2 and 3.

The maximum period length found is stored and updated whenever a fraction with a longer period is encountered. If two or more fractions have the same maximum period, the specified tie-breaking rules are applied: the fraction with the largest denominator is preferred, followed by the largest numerator if the denominators are equal. Once all eligible fractions have been examined, the stored values give the maximum period length and the corresponding fraction.

5.1 Code Implementation and Validation

The search was implemented, as seen in A.3, using nested `for` loops. The outer loop iterates through the eligible denominators, while the inner loop considers all valid numerators for each denominator. For every fraction, the period lengths and digit vectors returned by the two algorithms are compared. If either the period lengths or the digit vectors differ, the program produces an error.

After the search is complete, `maxPairs` is sorted by denominator and then numerator. The final row therefore identifies the pair with the largest denominator and, in the event of a tie, the largest numerator. The recurring digits of this selected fraction are then generated using `RecFrac1` and displayed alongside the maximum period and all pairs achieving it.

The implementation was validated by comparing the outputs of `RecFrac1` and `RecFrac2` for every fraction considered. Both the period lengths and the recurring digit vectors were required to agree; otherwise, the program would terminate with an error. No disagreements were reported during the successful execution of the program.

5.2 Results and Further Investigation

The search identified a maximum recurring period of 708 digits. This maximum was achieved by all 708 fractions with a denominator of 709 and numerators from 1 to 708. The complete set of pairs can be expressed as

$$\{(r, 709) : r = 1, 2, \ldots, 708\}.$$

According to the specified tie-breaking rules, the selected fraction was $\frac{708}{709}$, as it has the largest numerator among the fractions achieving the maximum period.

Finally, the recurring digits of $\frac{708}{709}$ were generated and displayed by the program. The returned period length of 708 agreed with the maximum identified by the search. The program output the complete recurring digit sequence shown in A.4.

Why is 708 special? This result has a number-theoretic explanation: 709 is a prime number, and the period of $\frac{1}{709}$ is the maximum possible value of $709 - 1 = 708$. Such primes are known as full reptend primes [2] in base 10, for which 10 has a multiplicative order $p - 1$ modulo the prime p. A possible extension would be to investigate a number-theoretic approach based on modular exponentiation and multiplicative order. This could determine period lengths without generating every recurring digit, potentially reducing the computation required when only the period is needed.

A MATLAB Code

A.1 RecFrac1

Listing 1: MATLAB implementation of RecFrac1

```
function [k,a] = RecFrac1(p,q)
% Recfrac1 uses Algorithm 1 and returns the recurring digits in the
% decimal expansion of p/q, among all pairs of positive integers
% (p, q) such that p<q and the last digit of q is a 9.

    % Input validation

    % Produces an error if the input of p is not a positive integer
    if ~isnumeric(p) || ~isreal(p) || ~isscalar(p) || ...
            ~isfinite(p) || p <= 0 || fix(p) ~= p
        error('p must be a positive integer.')
    end

    % Similarly for q
    if ~isnumeric(q) || ~isreal(q) || ~isscalar(q) || ...
            ~isfinite(q) || q <= 0 || fix(q) ~= q
        error('q must be a positive integer.')
    end

    % Produces an error if p is larger or equal to q or if the last digit
    % of q is not 9
    if p >= q
        error('p must be smaller than q.')
    end

    if mod(q,10) ~= 9
        error('the last digit of q must be 9.')
    end

    % Store initial remainder for stopping condition
    p0 = p;
    a = [];

    while true

        digit = floor(10*p/q);
        a(end+1) = digit;
        p = 10*p - digit*q;

        % Stopping condition
        if p == p0
            break
        end

```

```
    end

    k = length(a);
end
```

A.2 RecFrac2

Listing 2: MATLAB implementation of RecFrac2

```
function [k,a] = RecFrac2(p,q)
% % RecFrac2 uses Algorithm 2 and returns the recurring digits in the
% decimal expansion of p / q , among all pairs of
% positive integers ( p , q ) such that p < q and the last digit of q is a 9.

    % Input validation

    % Produces an error if the input of p is not a positive integer
    if ~isnumeric(p) || ~isreal(p) || ~isscalar(p) || ...
            ~isfinite(p) || p <= 0 || fix(p) ~= p
        error('p must be a positive integer.')
    end

    % Similarly for q
    if ~isnumeric(q) || ~isreal(q) || ~isscalar(q) || ...
            ~isfinite(q) || q <= 0 || fix(q) ~= q
        error('q must be a positive integer.')
    end

    % Produces an error if p is larger or equal to q or if the last digit
    % of q is not 9
    if p >= q
        error('p must be smaller than q.')
    end

    if mod(q,10) ~= 9
        error('the last digit of q must be 9.')
    end

    % Calculate the divisor
    d = (q - 9)/10 + 1;

    % Store the starting number and initialise the digits
    n = p;
    seen = [];
    a = [];

    % Continue until the current number has appeared before
    while ~ismember(n, seen)
        seen(end+1) = n;
```

```
41
42          digit = floor(n/d);
43          remainder = mod(n,d);
44
45          a(end+1) = digit;
46
47      % If the remainder is zero, the next number is just the digit
48          if remainder == 0
49              n = digit;
50          else
51              n = 10*remainder + digit;
52          end
53      end
54
55      k = length(a);
56  end
```

A.3 Maximum Period

Listing 3: MATLAB code to find the maximum period

```
1  %% Task 3: Find the maximum period
2
3  % Initialise the largest period found so far and list of pairs
4  maxPeriod = 0;
5  maxPairs = [];
6
7  % Consider denominators ending in 9 up to 749
8  for s = 9:10:749
9
10     % Consider every positive numerator smaller than s
11     for r = 1:s-1
12
13         [k1, a1] = RecFrac1(r,s);
14
15         [k2, a2] = RecFrac2(r,s);
16
17         % Check that both algorithms agree
18         if k1 ~= k2 || ~isequal(a1, a2)
19             error('The algorithms disagree for r=%d, s=%d.', r, s);
20         end
21
22         % Update the maximum period and the list of matching pairs
23         if k1 > maxPeriod
24             maxPeriod = k1;
25             maxPairs = [r, s];
26
27         elseif k1 == maxPeriod
28             maxPairs = [maxPairs; r, s];
```

```
        end
    end
end

% Display results
% Sort by denominator first, then numerator
maxPairs = sortrows(maxPairs, [2, 1]);

% Display the maximum period and all pairs that achieve it
fprintf('Maximum period length: %d\n', maxPeriod);
fprintf('All pairs (r,s) with the maximum period:\n');
disp(maxPairs);

% Select the pair with the largest denominator,
% then the largest numerator if there is a tie
chosenPair = maxPairs(end, :);
rBest = chosenPair(1);
sBest = chosenPair(2);

% Calculate and display the recurring digits
[~, bestDigits] = RecFrac1(rBest, sBest);

fprintf('Selected fraction: %d/%d\n', rBest, sBest);
fprintf('Recurring digits: 0.');
fprintf('%d', bestDigits);
fprintf('\n')
```

A.4 Recurring Digits of the Selected Fraction

The recurring decimal expansion of $\frac{708}{709}$ has a period of 708 digits. The digits are displayed below.

```
998589562764456981664315937940761636107193229901269393511988716502115655853314 5
275035260930888575458392101551480959097320169252468265162200282087447108603667 1
368124118476727785613540197461212976022566995768688293370944992947813822284908 3
215796897038081805359661495063469675599435825105782792665726375176304654442877 2
919605077574047954866008462623413258110014104372355430183356840620592383638928 0
677009873060648801128349788434414668547249647390691114245416078984485190409026 7
983074753173483779971791255289139633286318758815232722143864598025387870239774 3
300423131170662905500705218617771509167842031029619181946403385049365303244005 6
417489421720733427362482369534555712270803949224259520451339915373765867418 8
```

References

[1] Simon Stevin. *De Thiende, 1585 : Facsimile*. Dutch Classics on History of Science ; 15. Brill, Leiden, 1965. ISBN 9789004614253. URL https://brill.com/view/title/25729.

[2] John H. Conway and Richard K. Guy. *The book of numbers*. Copernicus, New York, 1996. ISBN 038797993X.